Inventory Management System

Project report

Field	Detail
Application	Inventory management for small and mid-size operations
Stack	Python 3.11+, Flask 3.1, SQLAlchemy, Jinja
Databases	SQLite for local work, PostgreSQL 16 in the container
Interface	Server-rendered HTML, four languages
Interface languages	English, Finnish, Simplified Chinese, Japanese
Interfaces exposed	86 routes, including 11 JSON endpoints
Code size	12,800 lines: 6,017 Python, 2,320 locale data, 3,754 templates, 709 CSS and JS
Automated checks	58 pytest tests, 22 barcode checks, 2 security probes
Package	Docker image with compose file, health check and named volumes

Figures taken from the route table, model list and test suite in this repository.

Contents

1. Executive summary
2. What the system does
3. Complete feature inventory
4. Features that set it apart
5. Advanced behaviour, in detail
6. Architecture and code layout
7. Deployment process
8. Configuration reference
9. Security
10. Quality assurance
11. Limits and things not included
12. Going live

1. Executive summary

This is a working inventory system, built to be run rather than demonstrated. It tracks stock per product per warehouse with a full movement history, handles batch and expiry tracking, runs purchase and sales orders through partial receipts and partial shipments, and reports on valuation, ageing, dead stock and reordering. Four people can use it in four languages at once, and an API sits alongside the interface for anything that needs scripting.

Two decisions shape most of the rest of this report. First, every quantity change goes through one service function that updates the stock figure and writes the ledger row in the same transaction, so the two cannot drift apart. Second, the barcode renderer is written from scratch with no external library and no network access, because a warehouse that cannot print a scannable label offline cannot work.

The application was reviewed against itself before delivery. The review found two critical issues, both of which are fixed and now covered by regression tests. See section 9.

Measure	Value
Registered routes	86
Database models	17
Public service functions	41
Reports	9, each with CSV export
Translated locales	3, each with 653 strings; English is the source language
Line counts	6,017 Python, 2,320 locale data, 3,754 templates, 709 CSS and JS
Pytest suite	58 tests, all passing
Barcode checks	13 encoder, 9 decode round-trip
Security probe findings	0 in both probes
External runtime dependencies	7 pinned packages

2. What the system does

Stock control for a small or mid-size operation. It runs on a single machine or in one container, with no build step for the front end and no paid services anywhere in the stack.

2.1 Areas of the system

Area	What it covers
Products	SKU, barcode, category, supplier, unit, cost, price, reorder point, bin location
Stock	Quantity per product per warehouse, append-only movement history, weighted average cost
Batches	Lot numbers and expiry dates, with earliest-expiry-first allocation
Orders	Purchase orders and sales orders, both with partial completion
Reports	Valuation, low stock, ABC, ageing, dead stock, expiry, reorder, supplier spend, audit
Scanning	Barcode scan desk with camera scanning and quick receive or issue
Access	Three roles, enforced per route, with an audit trail of who changed what
Language	English, Finnish, Chinese and Japanese with locale-aware formatting

2.2 The path a quantity takes

Nothing in the views changes a stock figure. A receipt, an issue, an adjustment and a transfer each call one function in services.py. That function writes the new on-hand quantity and appends a movement row inside a single database transaction, so a failure halfway through leaves neither change behind.

```
receive_stock(...)    goods in, recalculates weighted average cost
issue_stock(...)      goods out, allocates batches earliest expiry first
adjust_stock(...)     cycle count, records the difference as its own
movement
transfer_stock(...)   moves the exact batches taken from the source
warehouse
```

This is also what makes the audit trail complete. Every movement carries the user, the source address and a note, because the row is created by the same call that changes the stock.

3. Complete feature inventory

3.1 Stock control

- Stock held per product per warehouse
- Append-only movement ledger covering receipts, issues, adjustments, transfers and returns
- Weighted average cost recalculated on every receipt, so valuation reflects what was actually paid
- Cycle counts record the difference rather than overwriting history
- Stock cannot go negative unless an administrator explicitly permits it
- Multi-warehouse transfers that carry the batch detail with them
- Customer returns posted back through the same ledger
- Movement history filterable by product and exportable to CSV

3.2 Batch and expiry tracking

- Products flagged for batch tracking with a shelf life
- Receipts capture batch number and expiry date
- Issues draw the earliest expiry first, with undated batches deliberately used last
- Sales shipments and warehouse transfers allocate the same way
- Expiry watch on the dashboard and a dedicated report
- Expired and upcoming stock valued at cost
- Webhook events when stock is low, out of stock, expiring or adjusted

3.3 Ordering

- Purchase orders through draft, partial receipt, full receipt and cancel
- Receipts post real stock, so the order and the stock level cannot disagree
- Sales orders through draft, partial fulfilment and full fulfilment
- Short shipments refused with a message naming the shortfall
- One-click reordering that drafts a purchase order per supplier
- Reorder points computed from measured demand rather than a typed number
- Document numbers in the form PO-2026-00001, allocated atomically

3.4 Insight

- Dashboard with stock value, units on hand, low stock, expiry watch, open orders and charts
- Nine reports, every one of which exports to CSV
- ABC analysis by value contribution

- Stock ageing against a configurable review period
- Dead stock identified by days since last movement
- Supplier spend and lead-time performance
- Consumption series feeding the reorder calculation

3.5 Operations

- Barcode scan desk with camera scanning, handheld scanner support and manual entry
- Quick receive and issue from the scan screen
- CSV import with per-row error reporting
- CSV export from products, movements and both order types
- Printable shelf labels carrying scannable EAN-13 and Code128 codes
- Per-product barcode label with a print button
- Barcode matching that accepts the forms a real reader returns

3.6 Access and audit

- Three roles: admin, manager and staff
- Role checks enforced on the server for every route, not only in the interface
- Audit log of who changed what, with the source address
- Sign-in lockout for both the account and the address after repeated failures
- Profile page with per-user language preference and password change
- First-run setup page that closes permanently once an administrator exists

3.7 Languages

- English, Finnish, Simplified Chinese and Japanese
- Language switcher in the header with a per-user preference
- Configurable default for visitors who have not chosen
- Locale-aware number, money and date formatting
- 653 translated strings per locale, with a test that fails the build on any gap

3.8 JSON API

- Session cookie or X-API-Key header authentication
- Products: list, create, read, update
- Stock: receive and issue by SKU
- Reporting: summary, low stock, movements, lookup by barcode or SKU
- Health endpoint for container orchestration
- CSRF protection on every state-changing route, including the API

4. Features that set it apart

Most of the inventory systems on the market can count stock. The following are the ones that take work to get right, and they are where this one puts its effort.

Feature	Why it is unusual
Barcode renderer with no library and no network	EAN-13 and Code128 are drawn by 200 lines of local JavaScript. Labels print on a machine with no internet access, which is the normal condition on a warehouse network
Barcodes sized in millimetres	A label is the same physical size on screen and on paper, and the module width never drops below the 0.33mm floor EAN-13 readers need. Drawing at a rounded pixel pitch makes bars up to 18% wrong and stops them reading
Scan matching understands the code it printed	A twelve digit EAN body means the label encodes and prints thirteen digits, so a reader returns a value one digit longer than the one stored. The lookup accepts every form of the same number
Reorder points derived from your own consumption	Safety stock comes from the standard deviation of measured daily use scaled by the square root of lead time, at a service level you choose. The calculation is shown on the product page, not hidden
Earliest-expiry-first allocation, including transfers	Batch picking is not only applied to issues. Sales shipments and inter-warehouse transfers allocate the same way, and undated batches sort last so dated stock is always used first
Ledger and quantity written in one transaction	The stock figure and its movement row cannot diverge, because they are the same commit. The audit trail is complete as a side effect rather than as a separate task
Atomic document numbering	Order numbers are allocated with an UPDATE … RETURNING rather than a read then write, so parallel requests cannot be handed the same number. An earlier read-then-increment version lost six of eight concurrent purchase orders

Feature	Why it is unusual
Four languages with a build-breaking test	653 strings per locale, and a test that fails when any one is missing. A partially translated release cannot pass the suite
Works without a CDN for the parts that matter	Chart.js and the camera library come from a CDN. The barcode path does not, and handheld scanners work as keyboards regardless, so scanning survives an offline warehouse
Server-enforced roles on every route	Not merely hidden buttons. A staff account receives 403 on manager and admin routes, which the security probe verifies route by route

5. Advanced behaviour, in detail

5.1 Earliest-expiry-first batch allocation

On a batch-tracked product, an outbound quantity is split across the batches held at that warehouse. Batches are ordered with a null expiry date last, then by expiry date, then by identifier, so dated stock is always consumed before undated stock. The allocation returns a plan listing which batch gave up how many units, and that plan is what the movement rows record.

The consequence is that perishable stock does not sit while newer stock ships, and that a warehouse transfer carries the same lot discipline as an issue.

5.2 Reorder recommendation from measured demand

A typed reorder point goes stale the moment demand changes. This system derives one from the product's own consumption history over a 90 day window.

```
average daily use   = units consumed over the window / window days
safety stock        = z x standard deviation of daily use x square root of
lead time
reorder point       = ceiling(average daily use x lead time + safety stock)
reorder quantity    = ceiling(average daily use x (lead time + review
period))
```

The service factor z comes from a configurable service level: 0.674 at 50%, 0.842 at 80%, 1.282 at 90%, 1.645 at 95%, 2.054 at 98% and 2.326 at 99%. Lead time is taken from the supplier when there is one, otherwise from the system default of 7 days. A product with no recorded consumption produces no recommendation rather than a guess.

5.3 Weighted average cost

Each receipt recalculates the average cost per unit as the total value divided by the total quantity on hand, with the receipt's own cost folded in. Issues do not change the average. This is what the valuation report reads, so the figure on the dashboard and the figure in the accounts come from the same number.

5.4 Barcode encoding

The encoder picks a symbology from the value. Twelve or thirteen digits become EAN-13, with the check digit computed and printed. Anything else becomes Code128-B. Values outside the printable range are refused rather than drawn incorrectly, and the page falls back to showing the code as text.

Two rendering decisions make the difference between a label that reads and one that does not:

- Size is computed in millimetres and converted to whole device pixels per module, so every bar and every gap keeps its true proportion.
- A minimum module width of 0.33mm is enforced, even when the requested width would have produced something thinner.
- Quiet zones of 9 modules for EAN-13 and 12 for Code128 are drawn, with a white background behind the bars.
- The printed digit line appears under EAN-13 bars because a reader or a human may use it.

5.5 Barcode scan matching

An EAN-13 always encodes thirteen digits. When a product stores twelve, the label prints thirteen, so the reader returns a value one digit longer than the stored one. An exact comparison fails, which is the single most common reason a working label reports no match.

Normalisation accepts all of these for the same product:

- the value exactly as stored
- the value with spaces or dashes removed
- the twelve digit body and the full thirteen digit EAN, in either direction
- the zero-padded thirteen digit form and its twelve digit form

Digit stripping only applies to values that were numeric to begin with, so an alphanumeric code such as PCB-001 never collapses into a stray run of digits. Saving the same number on two products in different forms is refused, which stops two labels from resolving to the same record.

5.6 Order lifecycles

Order type	States	Effect on stock
Purchase order	draft, partial, received, cancelled	A receipt posts real stock through receive_stock
Sales order	draft, partial, fulfilled, cancelled	A shipment allocates batches earliest expiry first

A shipment is refused when there is not enough stock, with a message naming what is short. Cancellations do not touch stock because a cancellation means nothing was received or shipped.

5.7 Atomic document numbering

Document numbers come from a counter table. The increment is a single statement with a RETURNING clause, so the database serialises it. A read-then-write version was in place first and lost six of eight concurrent purchase orders to duplicate numbers; the atomic form was verified under eight parallel requests with eight unique numbers.

5.8 Webhook notifications

Four events are emitted to a configured URL:

Event	Fires when
stock.low	A product falls to or below its reorder point
stock.out_of_stock	A product reaches zero on hand
stock.batch_expiring	A batch enters the expiry warning window
stock.adjusted	Stock is changed by an adjustment or transfer

6. Architecture and code layout

```
inventory_system/
|-- app.py                 entry point, creates the application
|-- config.py              configuration objects, environment driven
|-- Dockerfile, docker-compose.yml, docker/entrypoint.sh
|-- requirements.txt
|-- docs/                  this documentation and the report
|-- scripts/               translation, security and browser-check tooling
|-- tests/                 pytest suite and the barcode suites
`-- inventory/
    |-- __init__.py        application factory, CLI commands, error handlers
    |-- extensions.py      db, login manager, CSRF
    |-- models.py          17 models
    |-- services.py        all stock rules and report aggregates
    |-- notifier.py        webhook delivery
    |-- utils.py           parsing, formatting, barcode matching, CSV
    |-- decorators.py      role guards
    |-- seed.py            administrator and four sample products
    |-- blueprints/        one module per area of the interface
    |-- templates/         Jinja templates
    |-- static/            stylesheet, app.js, barcode.js
    `-- i18n/              locale resolution, formatting, catalogues
```

The split matters more than the file list. Business rules live in services.py and the views call them. A report aggregate and the page that shows it read the same function, so a figure on the dashboard and the same figure in a report cannot disagree.

6.1 Dependencies

Package	Version	Purpose
Flask	3.1.3	web framework
Flask-SQLAlchemy	3.1.1	ORM session and models
Flask-Login	0.6.3	session authentication
Flask-WTF	1.2.2	CSRF protection
python-dotenv	1.1.1	reads .env
email-validator	2.3.0	email field validation
gunicorn	23.0.0	production WSGI server
psycopg[binary]	3.3.6	PostgreSQL driver, only when the URL points at postgres
pytest	8.4.2	development and test only

No front-end build step exists. There is no bundler, no npm dependency and no compiled asset pipeline, so a deployment is a copy of the repository.

7. Deployment process

7.1 Method A: Docker with Compose (recommended)

Needs Docker with the Compose plugin.

```
cp .env.docker.example .env

# edit .env and set two values
python3 -c "import secrets; print(secrets.token_urlsafe(48))"   # SECRET_KEY
python3 -c "import secrets; print(secrets.token_urlsafe(24))"   #
POSTGRES_PASSWORD

docker compose up -d --build
docker compose exec app flask --app app seed    # first run only
```

The application comes up on port 8000. Compose starts it under gunicorn with three workers against PostgreSQL 16, wires a health check to /api/health, and creates named volumes for the database and for application data. The image runs as a non-root user and declares its own HEALTHCHECK, so docker inspect reports status even outside compose.

Command	What it does
docker compose logs -f app	follow the application log
docker compose restart app	restart without rebuilding
docker compose down	stop and keep the data
docker compose down -v	stop and delete the database
docker compose exec app flask --app app check-translations	translation coverage
docker compose exec app flask --app app routes-list	list every registered route

The entrypoint waits for PostgreSQL by attempting a real connection rather than calling pg_isready, which is not present in the runtime image. It waits up to 60 seconds and then starts gunicorn.

7.2 Method B: SQLite inside Docker

```
# in .env
DATABASE_URL=sqlite:////app/data/inventory.db

# then delete the depends_on block from docker-compose.yml
```

SQLite lands on the app-data volume, so it survives restarts and rebuilds.

7.3 Method C: bare metal

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

export SECRET_KEY="$(python3 -c 'import secrets;
print(secrets.token_urlsafe(48))')"
export FLASK_CONFIG=production
export DATABASE_URL=postgresql+psycopg://user:password@host:5432/inventory

flask --app app init-db
gunicorn --bind 0.0.0.0:8000 --workers 3 --threads 4
"app:create_app('production')"
```

Put nginx or Caddy in front for TLS. Production mode already forces Secure cookies and keeps the debugger off, so plain HTTP sign-in is refused on purpose.

7.4 Deployment sequence

1. Install Python 3.11 or newer, or Docker with the Compose plugin.
2. Copy the repository and install the pinned dependencies.
3. Copy .env.docker.example to .env and set SECRET_KEY from a real secret generator.
4. Set POSTGRES_PASSWORD, or point DATABASE_URL at an existing database.
5. Start the stack: docker compose up -d --build.
6. Seed once: docker compose exec app flask --app app seed.
7. Confirm the health endpoint answers 200 on /api/health.
8. Change the administrator password, or seed with --no-demo and use the setup page.
9. Terminate TLS in front of the application.
10. Create staff accounts and give each one a role.
11. Take a first backup and confirm it restores.
12. Delete the sample products, categories and supplier.

7.5 Verification after deployment

Check	Command	Expected
Application is up	curl localhost:8000/api/health	200 with ok true
Schema is present	docker compose exec app flask --app app routes-list	86 routes
Translations complete	docker compose exec app flask --app app check-translations	no missing entries
Tests pass	pytest tests -q	58 passed

Check	Command	Expected
Probes clean	python scripts/security_probe.py	0 findings
Barcodes correct	node tests/barcode_decode.test.js	all checks passed
Interface behaves	python scripts/ui_check.py http://localhost:8000	no layout problems

7.6 Updates

```
docker compose exec app flask --app app init-db     # creates anything new,
keeps data
git pull
docker compose up -d --build
```

Existing tables are never dropped and adding a column is handled automatically. Anything structural should be tried on a copy of the database first, which the backup command provides.

8. Configuration reference

8.1 Environment variables

Variable	Default	Notes
SECRET_KEY	generated into instance/secret_key	set explicitly in production
DATABASE_URL	sqlite:///inventory.db	PostgreSQL works unchanged
FLASK_CONFIG	default	development, production or testing
DEFAULT_LOCALE	en	language for a visitor who has not chosen
ITEMS_PER_PAGE	20	rows per list page
SESSION_DAYS	7	session lifetime
SESSION_COOKIE_SECURE	false, true in production	leave on in production
FLASK_DEBUG	false	never enable on a reachable host
MAX_UPLOAD_MB	5	CSV import ceiling
MAX_LINE_QUANTITY	1000000	cap on one stock line
LOGIN_MAX_ATTEMPTS	5	failures before lockout
LOGIN_LOCKOUT_MINUTES	15	lockout window
TRUST_PROXY_HEADERS	false	true only behind a proxy you control
ALLOW_PRIVATE_WEBHOOKS	false	re-opens SSRF, leave false
APP_PORT	8000	host port published by compose
WEB_CONCURRENCY	3	gunicorn workers in the container

8.2 Settings stored in the database

These are edited at /settings, so they survive a redeploy without a restart.

Key	Default	Purpose
company_name	Inventory System	header and printed output
currency_symbol	$	used by the money formatter
low_stock_threshold	10	fallback when a product has no reorder point

Key	Default	Purpose
default_lead_time_days	7	used when a product has no supplier
allow_negative_stock	false	whether issues may take stock below zero
default_locale	en	language before a user chooses
service_level	95	safety stock service factor
review_period_days	30	stock ageing window and reorder cover
expiry_warning_days	90	how far ahead the expiry watch looks
notify_webhook_url	empty	destination for notifications
api_key	unset	authenticates the JSON API

8.3 CLI commands

```
flask --app app run-setup            # schema, settings and sample data
flask --app app init-db              # schema and settings only
flask --app app seed [--no-demo]     # users and sample data
flask --app app check-translations   # report untranslated strings
flask --app app routes-list          # every registered route
flask --app app shell                # db.session and all models pre-
imported
```

9. Security

The codebase was reviewed against itself using source inspection and two automated probe scripts run against a throwaway database. Both probes report zero findings. The full list of what was found and fixed follows.

9.1 Fixed issues

Severity	Issue	Fix
Critical	Session forgery with the shipped SECRET_KEY	Key read from the environment, or generated once into instance/secret_key with mode 600
Critical	Debug mode on by default, exposing the Werkzeug console	DEBUG defaults to False; enable deliberately with FLASK_DEBUG
High	No brute-force protection on sign-in	Account and address both lock for 15 minutes after 5 failures
Medium	SSRF through the webhook setting	URLs must be public; loopback, private, link-local and reserved ranges refused
Medium	Missing security headers	nosniff, SAMEORIGIN, referrer, COOP, permissions policy and CSP on every page
Medium	Unbounded file upload	Capped at 5 MB by default
Medium	First-run setup race	Setup returns 403 once any user exists; an installed_at marker breaks the tie
Medium	No ceiling on quantities	MAX_LINE_QUANTITY enforced on receipts, issues and transfers
Low	CSV export formula injection	Cells beginning with = + - @ are prefixed with an apostrophe
Low	Login timing reveals account existence	Unknown identities are verified against a dummy hash
Low	Audit log trusted a forgeable header	X-Forwarded-For read only when TRUST_PROXY_HEADERS is on

9.2 Checked and not exploitable

Area	Result
SQL injection	every value bound; eight payloads across eight endpoints produced no error and a tautology did not widen results
XSS	script tags in product names, movement notes and the search box are escaped on render
CSRF	every state-changing route returns 400 without a token, including logout and the API
Privilege escalation	staff get 403 on admin and manager routes; managers get 403 on user and settings routes
IDOR	no cross-record access; records are fetched by primary key and filtered by rights
Open redirect	protocol-relative, absolute, backslash and javascript: targets all rejected
Path traversal	a CSV named ../../../etc/passwd.csv wrote nothing outside the application
Session fixation	values written before sign-in do not survive it
Cookie flags	HttpOnly and SameSite=Lax always; Secure forced in production
Secrets in output	the API never returns password hashes; error pages show no tracebacks

9.3 Known limits

- The content security policy allows unsafe-inline for scripts because templates carry small inline blocks. Tightening it needs nonces threaded through the base template.
- Login throttling counts against the address and the account name. A distributed attacker spread across many addresses is slowed but not stopped; put a rate limiter in front of the application for that.
- The webhook guard resolves DNS once per send, so a hostname that resolves to a public address at check time and a private one at connect time is not covered.

10. Quality assurance

10.1 Test suites

```
pytest tests -q                    # 58 tests
node tests/barcode.test.js         # 13 encoder checks
node tests/barcode_decode.test.js  # 9 decode round-trip checks
python scripts/security_probe.py   # 0 findings
python scripts/security_probe2.py  # 0 findings
```

Suite	Covers
Sign-in and roles	setup, lockout, timing, role restrictions on every route
Stock maths	weighted average, negative stock policy, batch allocation, transfers
Batch and expiry	FEFO picking, expiry classification, expiry webhooks
Reordering	safety stock calculation, suggested quantities, generated orders
Orders	purchase and sales lifecycles, partial completion, short-shipment refusal
API	authentication, CSRF, CRUD, barcode lookup
Internationalisation	every string in every locale, locale-aware formatting
Barcode matching	12 and 13 digit forms, separators, zero padding, duplicate refusal
Rendering	every main page in all four languages
Barcode encoder	module strings against a reference encoding
Barcode decoder	render, read the pixels back, compare, verify the Code128 checksum

10.2 The decode round trip

The barcode encoder and the decoder were written from opposite directions. The encoder walks the EAN and Code128 tables forwards to produce a module string. The decoder is handed rendered pixels, collapses them into run lengths, converts those to modules and reconstructs the value the way a laser scanner would.

A mistake in either one shows up as a failed round trip rather than as a barcode that looks plausible. The decoder also verifies the Code128 checksum, which proves the

symbol boundaries were located correctly, and asserts that every run is a whole number of modules, which is the property that keeps bar proportions true.

10.3 Browser checks

scripts/ui_check.py drives a headless Chromium against a running instance and reports what a person would see: whether a toast is positioned above its card rather than beside it, whether a barcode canvas actually has ink on it, and whether the scan desk resolves a code. It exits non-zero on a layout problem.

```
python scripts/ui_check.py http://127.0.0.1:5000 --out /tmp/shots
python scripts/ui_check.py http://127.0.0.1:8000 --out /tmp/shots --admin-
password 'Admin@12345'
```

11. Limits and things not included

- Serial-number tracking. Batches are tracked by lot and expiry, not by individual unit.
- Manufacturing and bills of material.
- Multi-currency. One currency is configured system-wide.
- Purchase approval workflow. A manager can raise and receive an order without a second signature.
- Demand forecasting. Reordering is driven by measured consumption rather than a forecast.
- A dedicated label printer driver. Labels print through the browser, which needs print scaling left at 100%.
- Scheduled expiry alerts. Notifications fire when someone uses the system, not on a timer.

Two libraries are fetched from a CDN: Chart.js for the dashboard graphs and html5-qrcode for camera scanning. Neither affects stock operations, barcode labels or handheld scanning. Self-host the two files for a fully offline installation.

12. Going live

13. Change the administrator password, or seed with --no-demo and create the account through the setup page.
14. Set SECRET_KEY from a real secret generator.
15. Serve over HTTPS. Production mode already forces Secure cookies.
16. Keep SESSION_COOKIE_SECURE at its production default.
17. Only set TRUST_PROXY_HEADERS when a reverse proxy you control sets that header.
18. Only set ALLOW_PRIVATE_WEBHOOKS if the webhook must reach an internal address, and understand that it re-opens SSRF.
19. Leave FLASK_DEBUG unset.
20. Create the staff accounts you need and give each one a role.
21. Delete the sample products, categories and supplier.
22. Take a backup and confirm it restores.
23. Print a test label, scan it at the scan desk and confirm the product appears.